Data discovery and management tool - 'datadiscoverer'

Release 1.0

Kameswar Rao Modali
Karsten Peters-von Gehlen
Stephan Kindermann

Feb 21, 2024

CONTENTS:

CHAPTER

ONE

DATADISCOVERER

1.1 config module

This module contains the core structural elements of the data discovery tool. The core structural elements consists of the names of applications, HPC centers, ESMs etc.The names of the file types produced by each application and their extensions.Apart from that certain data structures to hold these infos.

class `config.configDatadiscoverer`

Bases: `object`

Class to hold the various paramters to drive the datadiscoverer.

`activeConfig = None`

`appDataSrcNameFileExt`

List of the file extensions.

Type
list[str]

`appDataSrcNames`

List of the file types. Names of the different types of data sources / files the applications can produce.

Type
list[str]

`appDescriptionInfo`

Dictionary of the description for each of the apps. Data structure to store the description, provider etc corresponding to each app.

Type
dict[str]

`appNames`

List of the application names. Names of the applications using the GSV data and producing respective application specific data.

Type
list[str]

`dataPath`

Path to location of the data produced by the applications.

Type
str

`esmNames`

List of the ESMs. Names of the Earth System Models used performing the simulations and producing the GSV data.

Type
list[str]

`getDataPath()`

Get the application data path.

For setting the location where the apps store the data that is to be cataloged by the datadiscoverer.

Parameters
`None.` –

Returns
Path to location of the data produced by the apps.

Return type
path

`getESMs()`

Set the ESM names.

For getting the ESM names from the configuration.

Parameters
`None.` –

Returns
List of the ESM names.

Return type
list[str]

`getHPCCenters()`

Get the HPC centers names.

For getting the HPC center names from the configuration.

Parameters
`None.` –

Returns
List of the HPC center names.

Return type
list[str]

`getInstallationPath()`

Get the datadiscoverer installation path.

For getting the location where the datadiscoverer is intalled on the disk.

Parameters
`None.` –

Returns
Path to datadiscoverer.

Return type
str

`getOutputPath()`

Get the output path.

For getting the location where the catalogs produced by datadiscoverer will be stored.

Parameters
`None.` –

Returns
Path to location of the catalogs produced by the datadiscoverer.

Return type
path

`getappDataSrcNameFileExt()`

Get the file name extension for the data sources.

For getting the file name extensions,for the various types of data sources that the data produced by the apps belongs to, from the the configuration.

Parameters
`None.` –

Returns
Dictionary of the data source names and the corresponding file name extensions.

Return type
appDataSrcNameFileExt

`getappDataSrcNames()`

Get the data source names.

For getting the data source names like 'netcdf', 'image', 'text' from the configuration.

Parameters
`None.` –

Returns
List of the data source names.

Return type
list[str]

`getappDataSrcs(`*app*`)`

Get the data sources produced by a given application.

For getting the data sources to which the data produced by a given application belongs to, from the configuration.

Parameters
`None.` –

Returns
List of data source names.

Return type
appDescInfo

`getappDescription(`*app*`)`

Get the description of a given application.

For getting the description about a given application from the configuration.

Parameters
`app` – Application name.

Returns
Application description.

Return type
str

`getappDescriptionInfo()`

Get the application description from the configuration.

For getting the application description info, provider etc from the configuration.

Parameters
`None.` –

Returns
Dictionary of the various parameters that describe the various apps.

Return type
appDescInfo

`getappMetadataKeys(`*app*`)`

Get the metadata keys for a given application.

For getting the metadata keys for the data produced by a given application from the configuration.

Parameters
`app` – Application name.

Returns
List of metadata keys.

Return type
str

`getappNames()`

Get the application names.

For getting the application names from the configuration.

Parameters
`None.` –

Returns
List of the application names.

Return type
list[str]

`getappProvider(`*app*`)`

Get the provider of a given application.

For getting the details about the provider of a given application from the configuration.

Parameters
`app` – Application name.

Returns
Details of the application provider.

Return type
str

`hpcCenters`

List of the HPC centers. Names of the HPC centers producing the data.

Type
list[str]

`installationPath`

Path to location of the data discovery and management tool on disk.

Type
str

`outputPath`

Path to location of the catalog prouced by data discovery and management tool.

Type
str

`setConfigfromJSON`(*jsonDict*)

Set the datadiscoverer configuration to the input JSON dictionary.

For setting the configuration parameters of the datadiscoverer from the input JSON dictionary.

Parameters
`jsonDict` – A JSON dictionary.

Returns
None.

`setDataPath`(*path*)

Set the application data path.

For setting the location where the apps store the data that is to be cataloged by the datadiscoverer.

Parameters
`path` – Path to location of the data produced by the apps.

Returns
None.

`setESMs`(*esmNames*)

Set the ESM names.

For updating the configuration with the ESM names.

Parameters
`list[str]` – List of the ESM names.

Returns
None.

`setHPCCenters`(*hpcCenters*)

Set the HPC centers names.

For updating the configuration with the HPC center names.

Parameters
`list[str]` – List of the HPC center names.

Returns
None.

`setInstallationPath`(*path*)

Set the datadiscoverer installation path.

For setting the location where the datadiscoverer is intalled on the disk to the configuration.

Parameters
`path` – Path to datadiscoverer.

Returns
None.

`setOutputPath`(*path*)

Set the output path.

For setting the location where the catalogs produced by datadiscoverer will be stored.

Parameters
`path` – Path to location of the catalogs produced by the datadiscoverer.

Returns
None.

`setappDataSrcNameFileExt`(*appDataSrcNameFileExt*)

Set the file name extension for the data sources.

For updating the configuration with the file name extensions for the various types of data sources that the data produced by the apps belongs to.

Parameters
`appDataSrcNameFileExt` – Dictionary of the data source names and the corresponding file name extensions.

Returns
None.

`setappDataSrcNames`(*appDataSrcNames*)

Set the data source names.

For updating the configuration with the data source names like 'netcdf', 'image', 'text'.

Parameters
`list[str]` – List of the data source names.

Returns
None.

`setappDescriptionInfo`(*appDescInfo*)

Set the application description info to the configuration.

For updating the configuration with the application description info, provider etc.

Parameters
`appDescInfo` – Dictionary of the various parameters that describe the various apps.

Returns
None.

`setappNames`(*appNamesList*)

Set the application names.

For updating the configuration with the application names.

Parameters
`list[str]` – List of the application names.

Returns
None.

`config.initializeDataDiscoverer(configFile)`

Initializing the datadiscoverer.

Initialize the datadiscoverer library configuration with the parameters provided in the input json file.

Parameters
configFile – JSON file with configuration parameters.

Returns:
curconfig - current configuration object.

1.2 intakeCuration module

This module contains the APIs for updating the intake catalog for each app and each of it's file types. These APIs need to be called by the app data curator for maintaining the intake catalog.

`intakeCuration.createAppIntakeCatalog()`

Create yaml files for the Apps.

For each of the Apps, create the yaml files containing the links to the yaml files for the ESMs .

Parameters
None –

Returns
None

`intakeCuration.createESMIntakeCatalog()`

Create yaml files for the ESMs.

For each of the ESMs, create the yaml files containing the links to the yaml files for the available sources .

Parameters
None –

Returns
None

`intakeCuration.createHPCIntakeCatalog()`

Create yaml files for the HPC centers.

For each of the HPC centers, create the yaml files containing the links to the yaml files for the Apps .

Parameters
None –

Returns
None

`intakeCuration.createIntakeCatalogSourcesForFileList(app, src, catFile, srcFileList)`

Create catalog for the sources for the inout file list.

For all the source files, create the intake counterparts and write out to the catalog file.

Parameters

- **app** – application Name
- **src** – source Name

- **catFile** – Catalog File Name.
- **srcFileList** – List of netcdf files.

Returns
None

`intakeCuration.createMasterIntakeCatalog(catalogFileName)`

Create master catalog file for the HPC centers.

Create the top level master catalog containing the links to the yaml files for the HPC centers.

Parameters
catalogFileName – String to hold the name of top level catalog file that would be created.

Returns
None

`intakeCuration.createSrcIntakeCatalog()`

Create yaml files for the sources.

For each of the app, check the available sources (file types) and create the corresponding yaml files.

Parameters
None –

Returns
None

`intakeCuration.intakeSrcCreator(app, src, srcFile, appMetadataKeys)`

Create yaml resource for the input source file.

For the input file of given source and app and metadata keys, prepare the intake object.

Parameters

- **app** – application Name
- **src** – source Name
- **catFile** – Catalog File Name.
- **srcFileList** – List of netcdf files.

Returns
intake catalog object.

Return type
intakeSrc

1.3 stacCuration module

This module contains the APIs for updating the STAC catalog for each app and each of it's file types. These APIs need to be called by the app data curator for maintaining the STAC catalog.

`stacCuration.createAppSTACCatalog()`

Create STAC catalog for the Apps.

For each of the Apps, create the STAC catalog containing the links to the STAC catalogs for the ESMs.

Parameters
None. –

Returns
None.

`stacCuration.`**`createESMSTACCatalog`**`()`

Create STAC catalog for the ESMs.

For each of the ESMs, create the STAC catalog containing the links to the STAC catalogs for the available sources.

Parameters
`None.` –

Returns
None.

`stacCuration.`**`createHPCSTACCatalog`**`()`

Create STAC catalog for the HPC centers.

For each of the HPC centers, create the STAC catalog containing the links to the STAC catalogs for the Apps.

Parameters
`None.` –

Returns
None.

`stacCuration.`**`createMasterSTACCatalog`**`()`

Create master STAC catalog file for the HPC centers.

Create the top level master STAC catalog containing the links to the STAC catalogs for the HPC centers.

Parameters
`None.` –

Returns
None.

`stacCuration.`**`createSTACSourcesForFileList`**`(`*app*, *src*, *srcCatalog*, *srcFileList*`)`

Create STAC items for the netcdf files in the input
file list of files.

For each of the netcdf files in the input file list, create the STAC item and save in the input catalog file.

Parameters

- **`app`** – Application Name.
- **`src`** – Source Name.
- **`catFile`** – Catalog File Name.
- **`srcFileList`** – List of netcdf files.

Returns
None

`stacCuration.`**`stacItemCreator`**`(`*app*, *src*, *srcFile*, *appMetadataKeys*, *itemId*`)`

Create stac item for the input source file.

For the input file of given source and app and metadata keys, prepare the STAC item.

Parameters

- **`app`** – application Name
- **`src`** – source Name

- **catFile** – Catalog File Name.
- **srcFileList** – List of netcdf files.

Returns
intake catalog object.

Return type
intakeSrc

1.4 utils module

This module contains certain utitlity functions like for fetching the files from a given location on disk etc.

utils.**getAppSrcFileList**(*app*, *src*, *esm*, *srcExtList*)

Create a list of files produced by the app, for a given file type.

Search the app data directory and create a list of all the files of a given file type like netcdf, jpeg etc.

Parameters

- **app** – Application Name.
- **src** – Source or file type name like netcdf, image, text.
- **esm** – Earth system model Name.
- **srcExtList** – The file suffix like *.nc, *.jpg, *.pdf, *.txt, *.csv.

Returns
srcFileList - a list.

CHAPTER

TWO

INDICES AND TABLES

- genindex
- modindex
- search

PYTHON MODULE INDEX

INDEX